

Augmenting Financial Analysis With AI Agents

Porter Jenkins
Brigham Young University
pjenkins@cs.byu.edu

Andrew hall
Brigham Young University
amh1124@byu.edu

Chuck Nadauld
Brigham Young University
cnadaul2@byu.edu

Abstract

This paper examines the application of artificial intelligence agents in financial forecasting and analysis, with particular focus on small to medium-sized businesses (SMBs). We compare traditional Software as a Service (SaaS) solutions with emerging AI agent frameworks for percent of sales forecasting tasks. Through experimentation, we demonstrate that AI agents using specialized tools can achieve better accuracy compared to traditional agentic workflows. Our findings show that AI agents are a promising solution for financial analysis previously available only to enterprises with dedicated finance teams.

Introduction

Financial analysts are a key part of company success because of their ability to analyze complex financial data, generate actionable insights, and inform strategic decision making. Their core responsibilities typically include budget preparation, financial forecasting, and project profitability evaluation. These activities have traditionally been facilitated by spreadsheet applications like Microsoft Excel, which allow analysts to create intricate financial models through formulaic relationships between variables.

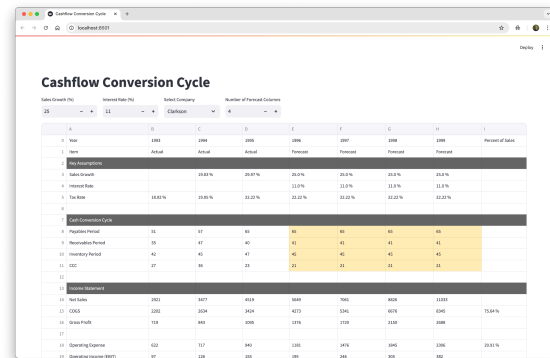
Recent advances in Large Language Models (LLMs) and AI agents have introduced the possibility of automating complex tasks that previously required human expertise. While large enterprises often have their own dedicated financial analysis teams, SMBs frequently lack the resources to allocate additional headcount. This resource gap creates an opportunity for AI agents to provide financial analysis capabilities at significantly reduced costs.

This paper investigates two approaches to addressing this need: traditional SaaS applications and emerging AI agent frameworks. We focus specifically on percent of sales forecasting (a common financial analysis technique) as a representative task to evaluate the efficacy of these approaches.

Software as a Service Solution

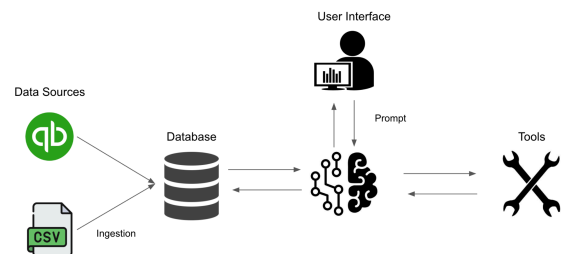
Financial forecasting has evolved significantly from manual calculations on paper spreadsheets to sophisticated software applications. The development of spreadsheet software has significantly increased the ease of financial modeling by enabling analysts to create dynamic relationships between variables.

Companies such as Oracle and NetSuite have even created dedicated financial planning and analysis (FP&A) platforms that are essentially more powerful versions of Microsoft Excel. While these platforms have an abundant set of features and customizability, they are often not adopted by SMBs due to the complexity of the software and the high price tag.



	2021	2022	2023	2024	2025	2026	2027	2028	2029	Percent of Sales
Total	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	
Accounts Receivable										
Accounts Payable										
Inventory										
Fixed Assets										
Equity										
Debt										
Operating Income										
Net Income										
EPS										
Dividends										
Free Cash Flow										
Operating Capital										
Working Capital										
Capital Expenditures										
Depreciation										
Amortization										
Goodwill Impairment										
Other Non-Operating Income										
Other Non-Operating Expenses										
Other Non-Operating Income/Expenses										

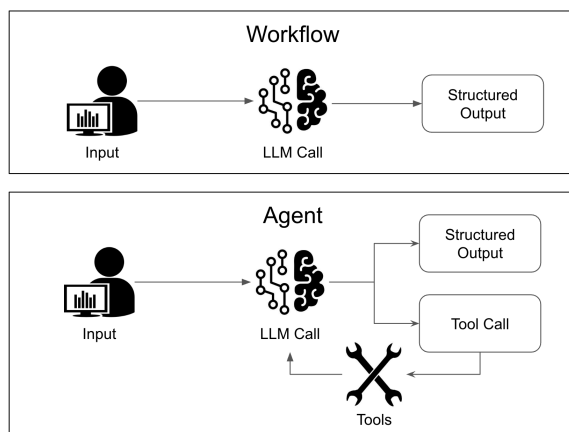
In order to gauge the feasibility of a standard SaaS application for SMBs we developed an application for percent of sales forecasting with SQLite for the database, vanilla Python for the backend, and Streamlit for the frontend. The system was designed to accept financial data from multiple sources such as the QuickBooks API and raw CSV files. This was made possible by creating data pipelines that turn our data sources into standardized SQL tables. A modular forecasting framework was also created with a base Spreadsheet class that mimics the functionality of traditional spreadsheet applications. This class provided methods for establishing value and formula cells as well as evaluating the entire spreadsheet when state changes. The application also allows users to control various parameters of the forecasting process, including the historical data range and number of years to forecast.



In the end we found that the standard SaaS application doesn't work very well since each additional forecasting input feature increases the complexity of the underlying codebase. We also realized that as the number of buttons and knobs increased, the front end application was no longer discernible from an ordinary spreadsheet. Increased customizability only reverted the software to its original form.

AI Agent Solution

AI agents that can perceive their environment, make decisions, and take actions to achieve specific goals have emerged as a promising framework for analytical applications. The distinction between structured workflows and flexible agent architectures presents a key design consideration in this solution. Workflows provide predictability and transparency with predetermined prompt sequences, while agents offer adaptability through self directed chain of thought. This tradeoff between structure and flexibility has significant implications for the effectiveness of this solution.



In order to determine the most efficient framework for an agentic solution we selected percent of sales forecasting as our target task due to its common use in financial analysis. This method involves forecasting a company's future sales based on historical growth and projecting other financial items (e.g., cost of goods sold, operating expenses) as percentages of the forecasted sales. This approach allows businesses to create forward looking financial statements that maintain linear relationships between line items.

We implemented and evaluated four distinct agent architectures: basic workflow, tools workflow, basic agent, and tools agent. For implementation, we utilized the Hugging Face smolagents Python

package as the core technology driving our agents and extended it with custom tools specific to percent of sales forecasting. These tools included data retrieval tools that allow the LLM to access the database and various forecasting tools such as a growth rate calculator. Each tool was implemented as a Python function with strict input validation and type checking. The workflows used three distinct prompts corresponding to the three main forecasting steps, while the agents were provided with a single comprehensive prompt describing the overall task.

Experiments

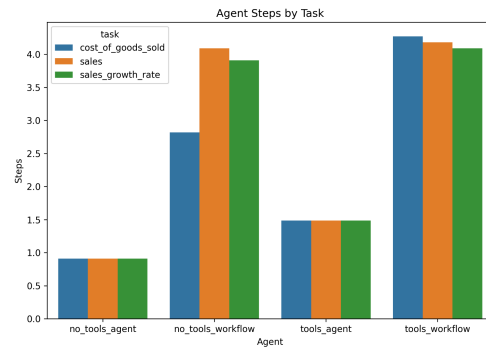
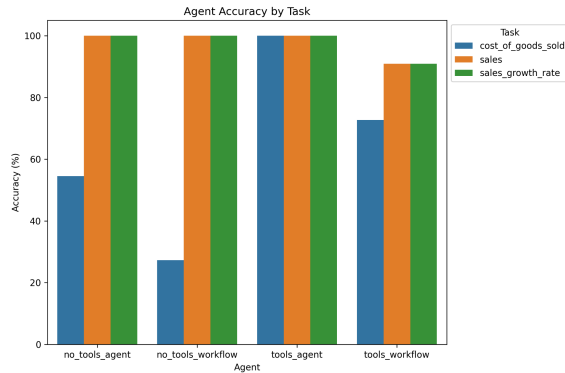
To evaluate the performance of each approach, we designed an experiment focusing on a simplified percent of sales forecasting task with three components:

1. Calculating the average historical sales growth rate
2. Forecasting future sales using the calculated growth rate
3. Projecting cost of goods sold as a percentage of forecasted sales

We used data from a Harvard business school case study surrounding Play Time Toy Co. for these experiments. Each agent architecture was tested with the same dataset and evaluation criteria. We conducted 15 trial runs for each architecture to account for potential variability in LLM outputs. Performance was evaluated based on a binary score for accuracy (1 if the result was correct to 2 decimal places, 0 otherwise). The underlying model for all agents was Qwen2.5-Coder-32B-Instruct.

Results

The Tools Agent achieved perfect accuracy across all tasks, while the Basic Agent performed nearly as well with perfect accuracy on all but the third task. The workflow based approaches demonstrated lower accuracy, with the Tools Workflow being the only agent to not score 100% on any of the tasks. The discrepancy in performance between agent and workflow architectures was most pronounced in the final costs of goods sold forecasting task. This was due to the cascading nature of the computations at hand with the output of previous prompts being implemented in the inputs of subsequent prompts. Interestingly, the addition of tools was beneficial for agents but not for workflows. This might be due to the excessive rigidity inherent in a static workflow with specific function calls.



In addition to measuring accuracy, we also measured the amount of time taken to perform each task, the number of input and output tokens used, and the number of steps taken by each agent. We find that the workflow architectures require more discrete steps to complete the tasks. This was expected given their predetermined sequence of operations. However, the agent architectures consumed significantly more tokens and execution time. This reflects the computational cost of higher accuracy, and demonstrates a clear tradeoff between performance and resource efficiency.

Conclusion

Our findings highlight fundamental differences between traditional SaaS and AI agent approaches to financial forecasting. The SaaS implementation provided a structured, predictable environment but required significant upfront development effort and became increasingly complex as additional features were added. The continuous addition of input parameters created a user experience that ultimately mimicked the complexity of the spreadsheet applications it was meant to replace. In contrast, the AI agent approach offered greater flexibility and adaptability with less implementation overhead. However, this flexibility comes with tradeoffs in predictability and resource efficiency. The addition of specialized tools significantly enhanced accuracy for agent architectures, but with substantial increases in token usage.

For SMBs facing resource constraints, AI agent approaches offer a promising middle ground for providing sophisticated analytical capabilities without the complexity of traditional enterprise FP&A software. As LLM capabilities continue to advance, we anticipate that AI agents will play an increasingly important role in democratizing access to sophisticated financial analysis capabilities, enabling businesses of all sizes to make more informed financial decisions.

